

Djinni-FB

Plataforma web para jugar rol de mesa online

Javier Granizo Aguilar

Grado Superior en DAW

Escuela de Arte de Granada · 2025-2026

Tutor: Manuel Prieto Macias

15 minutos de exposicion + preguntas del tribunal

Indice de la defensa

1. Introduccion. El problema y la solucion. (2 *min*)
2. Arquitectura y tecnologias. (3 *min*)
3. Modelo de datos. (2 *min*)
4. Demostracion en vivo. (6 *min*)
5. Conclusiones y trabajo futuro. (2 *min*)

1. Introduccion

El problema

Grupos que jugaban presencial dejan de coincidir.

Las herramientas existentes son potentes pero pesadas
(curva de aprendizaje alta, suscripcion, atadas a un sistema).

Lo que falta:

- Tablero compartido en tiempo real, ligero, en navegador.
- Hoja de personaje editable de verdad, no un PDF rellenable.
- Despliegue propio. Sin depender de la nube ajena.

La solucion

Djinni-FB: tablero virtual + hoja de personaje + chat, sincronizados por WebSocket, autoalojable con Docker.

Tres principios de diseno:

- **Director / jugador como roles por partida**, no por usuario global.
- **Tablero como capas**: fondo, jugador, GM, muros, niebla.
- **Tiempo real con autoridad en servidor**, no con CRDT en cliente.

Publico: grupos ya formados (3 a 6 personas) que quieren seguir jugando.

2. Arquitectura y tecnologías

Vision general

center

Cinco contenedores Docker. Un solo `docker compose up -d`.

Stack

Capa	Tecnologia
API REST	Symfony 8 + PHP 8.4
ORM	Doctrine 3.6
Auth	JWT (lexik/jwt-authentication-bundle)
WebSocket	Workerman 5 (proceso PHP de larga vida)
Base de datos	MariaDB 10.11
SPA	React 19 + Vite 7
Canvas	react-konva 19 / Konva 10
Estilos	Tailwind CSS 3.4
Despliegue	Docker Compose

Decisiones que conviene defender

Workerman frente a Mercure. El modelo "salas por partida + filtrado de capa GM antes de emitir" no se modela limpio en pub/sub. Workerman me permite mapear `gameId -> [conexiones]` en proceso.

React frente a Twig. El tablero hace cientos de eventos por segundo. Devolver fragmentos HTML es inviable. SPA con estado en cliente.

JWT frente a sesiones. SPA y API en distintos puertos.

JWT en cabecera evita CORS con cookies cross-site y se reusa en el handshake del WebSocket.

Seguridad implementada

- Hashing `auto` (Argon2id / Bcrypt segun PHP).
- JWT con RSA, claves generadas en primer arranque dentro del contenedor.
- `login_throttling` : 5 intentos / 15 min. Registro: 3 / h. Subidas: 30 / min.
- `UploadValidator` : MIME real, lista blanca, tope 5 MB, anti *path traversal*.
- `SessionAccessChecker` : pertenencia y rol director centralizados.
- Filtrado de tokens GM **en servidor y en cliente** (defensa en profundidad).
- Voters Symfony registrados para nuevos endpoints.

3. Modelo de datos

Entidades nucleo

center

Diagrama completo en Mermaid `erDiagram` en `memoria/diagramas/er.mmd` .

Decisiones de modelado

Rol por sesion, no por usuario. `UserGameSession.is_dm` decide quien dirige cada partida. Un mismo usuario puede ser director en una partida y jugador en otra. No usar `ROLE_ADMIN` global.

Datos espaciales en JSON. `Scene.fog_data` y `Scene.walls_data` son JSON. Comodo en cliente, dificil de auditar. Si lo rehiciera, los muros irian a tabla normalizada.

Hoja de personaje plana. ~50 columnas en `character_sheet`.
Sub-tablas `Attack`, `Ability`, `Inventory`, `Spell` solo para colecciones realmente variables.

4. Demostracion practica

Recorrido de demo (6 min)

1. Levantar la app: `docker compose up -d` (mostrar logs).
2. Login con `dm@djinni.local`.
3. Lista de partidas. Entrar en "Partida de demostracion".
4. Crear escena. Pintar muros y niebla.
5. Abrir segunda ventana con `jugador@djinni.local`.
6. Spawnear token. Arrastrar. Ver sincronizacion en vivo.
7. Editar hoja: ataque, tirada al chat publico y al privado.
8. Subir mapa de escena. Validacion MIME en accion.

Guion detallado en `demo_script.md`.

5. Conclusiones y trabajo futuro

Lecciones aprendidas

- **Dockerizar al final fue un error.** El primer día del TFG debería haber sido el `docker-compose.yaml`.
- **JSON en BD es comodo y mal auditable.** Modelar muros normalizados habria ahorrado *bugs* de regresion.
- **El roster crecio mas de lo previsto.** Es la pieza con mas valor anadido despues del tablero, y la mas dificil de explicar.

Limitaciones conocidas

- Sin suite de tests automatica. Solo verificacion manual.
- Workerman en un unico proceso. Estado de salas en memoria.
- Reconexion WebSocket pierde eventos entre desconexion y reconexion.
- Sin internacionalizacion. Interfaz solo en castellano.

Trabajo futuro

- Tests funcionales (`webTestCase`) y E2E (Playwright).
- Estado de salas en Redis para escalado horizontal.
- *Replay* de eventos al reconectar.
- `EditCharacterModal` y `VttBoard` partidos en subcomponentes.
- Importacion de hojas desde formatos comunes (D&D Beyond, Foundry).
- Sistema de macros para tiradas reutilizables.

Gracias

Codigo, memoria y manual de despliegue en el zip de entrega.

Preguntas.

Javier Granizo Aguilar · javiercelia123@gmail.com